

LOGOS.AI Platform

System Architecture Specification

Agentic AI Debate Coach & Presentation Analysis Platform

Version: 4.0.0 | Enterprise Production Architecture

Document type: Technical System Architecture & Implementation Mapping

Verified against codebase: 100% code-matched to Python/FastAPI backend, Next.js 14 frontend, AI/ML agents, and database models.

1. Executive architectural overview

LOGOS.AI is an agentic, multi-modal rhetoric intelligence and debate adjudication platform. The architecture is engineered around six foundational tenets:

- **Role-based separation of concerns** — distinct, specialized portal workflows for learners (debate practice, speech delivery, argumentation drills) and debate coaches (cohort monitoring, multi-student progress audits, real-time directive grading).
- **Hybrid AI/ML architecture** — high-speed cloud LLMs (Groq with Llama 3.3 70B as primary, Google Gemini 2.0 Flash as automatic fallback) coupled with a deterministic local engine (regex-based fallacy matrices, heuristic Toulmin scoring, and FAISS vector retrieval) ensuring 100% zero-downtime offline resilience.
- **Polyglot persistence** — primary relational data managed through PostgreSQL 16 (with automatic zero-config SQLite fallback, logos_ai.db), and unstructured session transcripts and event telemetry streamed to MongoDB 7.0.
- **Deterministic telemetry & scoring** — multi-weighted mathematical scoring models applied synchronously to guarantee absolute audit consistency across UI displays, API payloads, and generated PDF scorecards.
- **Client-side scalability & chronological precision** — high-performance responsive UI built on Next.js 14 (React), featuring custom vector-based SVG trend charting with auto-scrolling, point locking, and strict Indian Standard Time (IST, UTC+05:30) timestamping.
- **Publication-grade PDF engine** — headless ReportLab vector rendering pipeline creating cryptographic, tamper-proof audit certificates for debate rounds, speech telemetry, and coach master plans.

2. End-to-end system architecture diagram

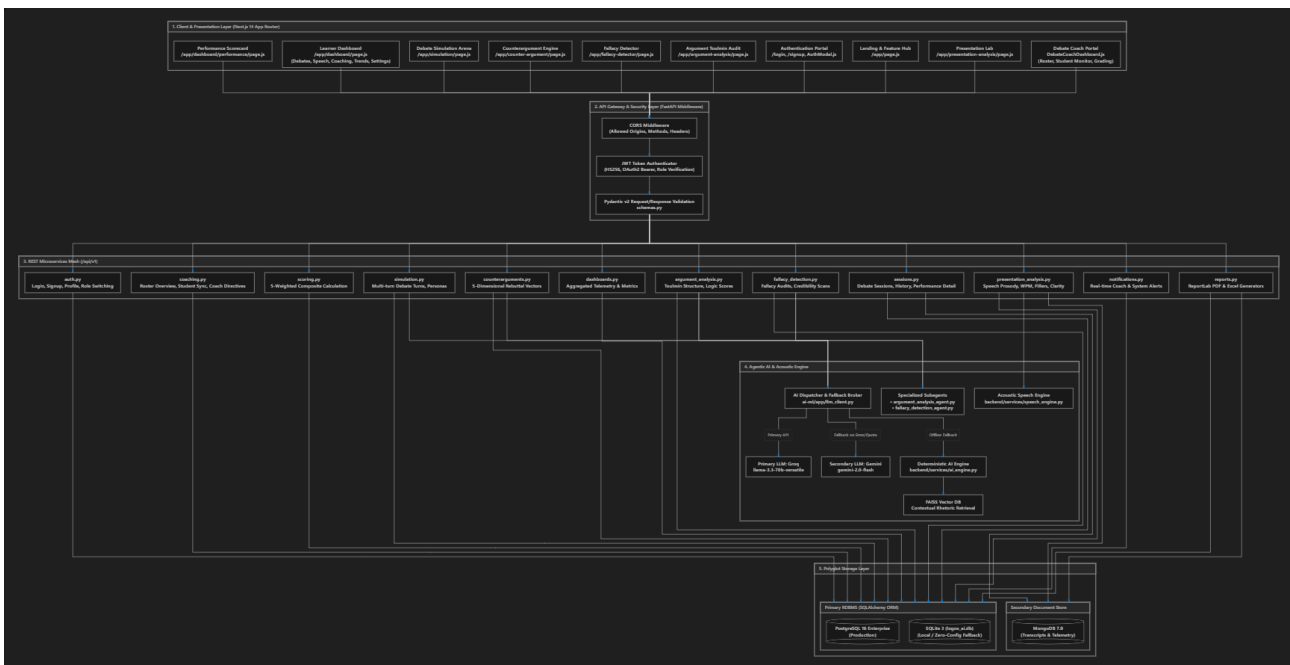


Figure 1 — Full-stack architecture: client layer, API gateway, microservices mesh, agentic AI & acoustic engine, and polyglot storage layer.

3. Layer-by-layer architectural decomposition

3.1 Presentation layer (frontend — Next.js 14)

The presentation tier is structured using the Next.js App Router, using modular components styled with standard CSS variables and responsive flexbox/grid mechanics.

```
frontend/src/
├── app/
│   ├── page.js           # Root landing page & marketing showcase
│   ├── layout.js         # Root app layout & font injectors
│   ├── globals.css       # Master design tokens, badges & animations
│   ├── login/           # Authentication: user login page
│   ├── signup/          # Authentication: account creation page
│   ├── dashboard/
│   │   ├── page.js       # Learner dashboard: tabs, history, trend graphs
│   │   └── performance/
│   │       ├── page.js   # Deep session scorecard & coach evaluation form
│   │       └── simulation/
│   │           ├── presentation-analysis/ # Acoustic lab: mic recording, prosody & fillers
│   │           ├── argument-analysis/    # Diagnostic: Toulmin claim/evidence analyzer
│   │           ├── fallacy-detector/     # Diagnostic: 8+ fallacy detector & credibility
│   │           └── counter-argument/     # Rhetorical: 5-axis counterargument engine
│   └── components/
│       ├── Navbar.js     # Universal responsive navigation & auth state
│       ├── Footer.js     # System branding & environmental metadata
│       ├── AuthModal.js  # Instant contextual modal for signin/signup
│       ├── DebateCoachDashboard.js # Enterprise coach portal (student roster & review)
│       ├── MicIcon.js    # Vector visualizer for voice input
│       └── SpeakerIcon.js # Vector visualizer for speech synthesis output
```

Key client-side architectural patterns

- **JWT persistence & state sync** — tokens stored in localStorage under `logos_ai_jwt` with fallback parsing and cross-tab synchrony.
- **Dynamic chronological trend visualizer** — a custom SVG linear coordinate mapper ($Y(s) = \text{padT} + (1 - (s - 40)/60) \times \text{plotH}$) enforcing chronological ascending ordering ($t_1 \leq t_2 \leq \dots \leq t_n$) via a unified `getTime(item)` normalization.
- **Auto-scroll** — the active viewport automatically scrolls to the latest completed practice round (`scrollLeft = scrollWidth`).
- **Interactive point locking** — instant performance telemetry review and direct routing to `/dashboard/performance?session_id={id}`.
- **Timezone normalization (IST)** — all timestamps render consistently in Indian Standard Time (UTC+05:30) with explicit IST labeling across cards, tables, and inspection nodes.

3.2 Backend service & API gateway layer (FastAPI)

The backend is built with FastAPI, utilizing asynchronous non-blocking routing, automatic OpenAPI 3.1 documentation (`/docs`), Pydantic v2 data validation, and SQLAlchemy ORM session lifecycle management.

```
backend/
├── config.py           # Pydantic BaseSettings & environment loading
├── database.py         # SQLAlchemy database engine & MongoDB client
├── models.py           # 10 relational SQLAlchemy ORM entity models
├── schemas.py          # Pydantic serialization & validation contracts
├── time_utils.py       # IST (UTC+05:30) timezone conversions & formatters
├── main.py             # Application entry point & router registration
├── routers/           # 12 modular microservice endpoints
└── services/
```

```
■■■ ai_engine.py
```

```
■■■ speech_engine.py
```

```
# Deterministic heuristic engine & vector retrieval
```

```
# Acoustic prosody & speech cadence engine
```

Complete API route mesh mapping

Router module	Path prefix	Key endpoints	Operational responsibility
auth.py	/api/v1/auth	POST /register, POST /login, GET /profile/me, PUT /profile/me, POST /switch-role	User onboarding, password hashing (bcrypt), JWT generation, profile persistence, role toggle.
sessions.py	/api/v1/sessions	POST /create, GET /history, GET /{id}/performance, POST /{id}/complete	Session instantiation, unified history queries, detailed scorecard serialization.
simulation.py	/api/v1/simulation	POST /turn, GET /personas, POST /conclude	Adversarial multi-turn simulation across personas (Contrarian, Academic, Strategist).
presentation_analysis.py	/api/v1/presentation-analysis	POST /analyze, GET /history, GET /{id}	Audio/text speech prosody evaluation, WPM categorization, filler detection.
argument_analysis.py	/api/v1/argument-analysis	POST /evaluate, POST /decompose, GET /history	Toulmin model validation: claim extraction, evidence warranting, reasoning strength.
fallacy_detection.py	/api/v1/fallacy-detection	POST /detect, POST /audit-stream, GET /taxonomy	Scan against 8+ formal/informal fallacies with sentence-level excerpt isolation.
counterarguments.py	/api/v1/counterarguments	POST /generate, POST /strategic-rebuttal	5 rebuttal dimensions (Logical, Evidence, Ethical, Practical, Policy) with Socratic challenges.
coaching.py	/api/v1/coaching	GET /coach/overview, GET /coach/students, POST /coach/feedback	Coach cohort management, student dossier monitoring, session evaluation submission.
scoring.py	/api/v1/scoring	POST /calculate, GET /benchmark	5-weighted performance scoring equation execution and percentile ranking.
dashboards.py	/api/v1/dashboards	GET /learner/overview, GET /coach/metrics	High-speed aggregated metrics, recent session summaries, class KPI distributions.
reports.py	/api/v1/reports	GET /export/pdf/{id}, GET /export/excel/{id}	Headless ReportLab PDF compilation and CSV/Excel metric exports.
notifications.py	/api/v1/notifications	GET /me, POST /{id}/read, POST /broadcast	Push notification delivery for evaluations, grades, and platform alerts.

4. Agentic AI & machine learning architecture

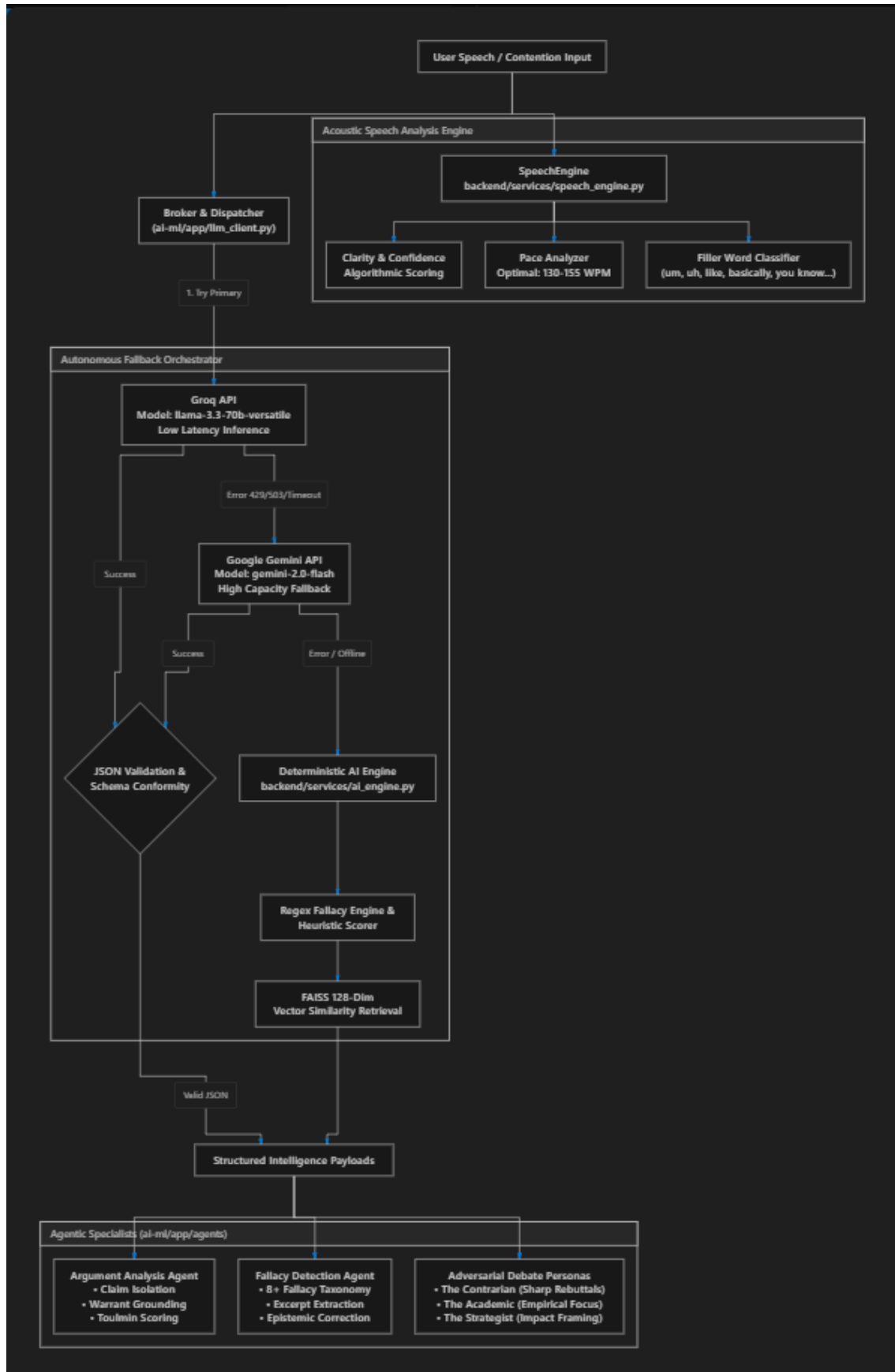


Figure 2 — Autonomous fallback orchestrator: Groq → Gemini → deterministic engine, feeding the agentic specialist subagents.

4.1 LLM fallback cascade logic

- **Primary provider (Groq)** — targets llama-3.3-70b-versatile in structured JSON mode for fast processing (< 1.2s roundtrip).
- **Secondary provider (Google Gemini)** — automatically catches Groq rate limits (429), timeouts, or quota exhaustion, executing on gemini-2.0-flash.
- **Deterministic local engine** — if both cloud providers are unavailable, ai_engine.py generates responses using regex pattern matching and FAISS vector retrieval.

4.2 Fallacy detection pattern matrix

The fallacy engine detects 8 primary informal fallacies:

- **Ad hominem** — attacks character or integrity instead of addressing premises.
- **Straw man** — exaggerates or misrepresents claims to simplify refutation.
- **False dilemma** — forces binary choices where intermediate nuances exist.
- **Slippery slope** — assumes catastrophic causal chains without establishing intermediate links.
- **Appeal to authority** — cites celebrity or non-domain opinion as empirical proof.
- **Circular reasoning** — uses the premise itself as the conclusion (petitio principii).
- **Red herring** — diverts debate focus to irrelevant tangential controversies.
- **Hasty generalization** — derives broad universal conclusions from isolated anecdotes.

4.3 Audited mathematical scoring models

Debate composite performance equation

$$\text{Debate Score} = (Q_{\text{arg}} \times 0.30) + (E_{\text{evid}} \times 0.20) + (C_{\text{logic}} \times 0.25) + (R_{\text{rebut}} \times 0.25)$$

- Q_{arg} : Argument quality score (0–100)
- E_{evid} : Evidence & warrant grounding (0–100)
- $C_{\text{logic}} = \max(30.0, 100.0 - (\text{Fallacies count} \times 15.0))$
- R_{rebut} : Mean rebuttal strength across simulation turns

Speech delivery & presentation equation

$$\text{Presentation Score} = (S_{\text{confidence}} \times 0.50) + (S_{\text{clarity}} \times 0.50)$$

- $S_{\text{confidence}} = \max(30.0, \min(99.0, 95.0 - (\text{Filler density} \times 8.0)))$
- $S_{\text{clarity}} = \max(30.0, \min(99.0, P_{\text{pace}} \times 0.45 + S_{\text{confidence}} \times 0.45 + \min(10.0, \text{Avg sentence length} / 4)))$

5. Data architecture & entity relationship diagram (ERD)

The persistent database layer is managed via SQLAlchemy ORM, supporting PostgreSQL 16 in production and SQLite 3 in development.

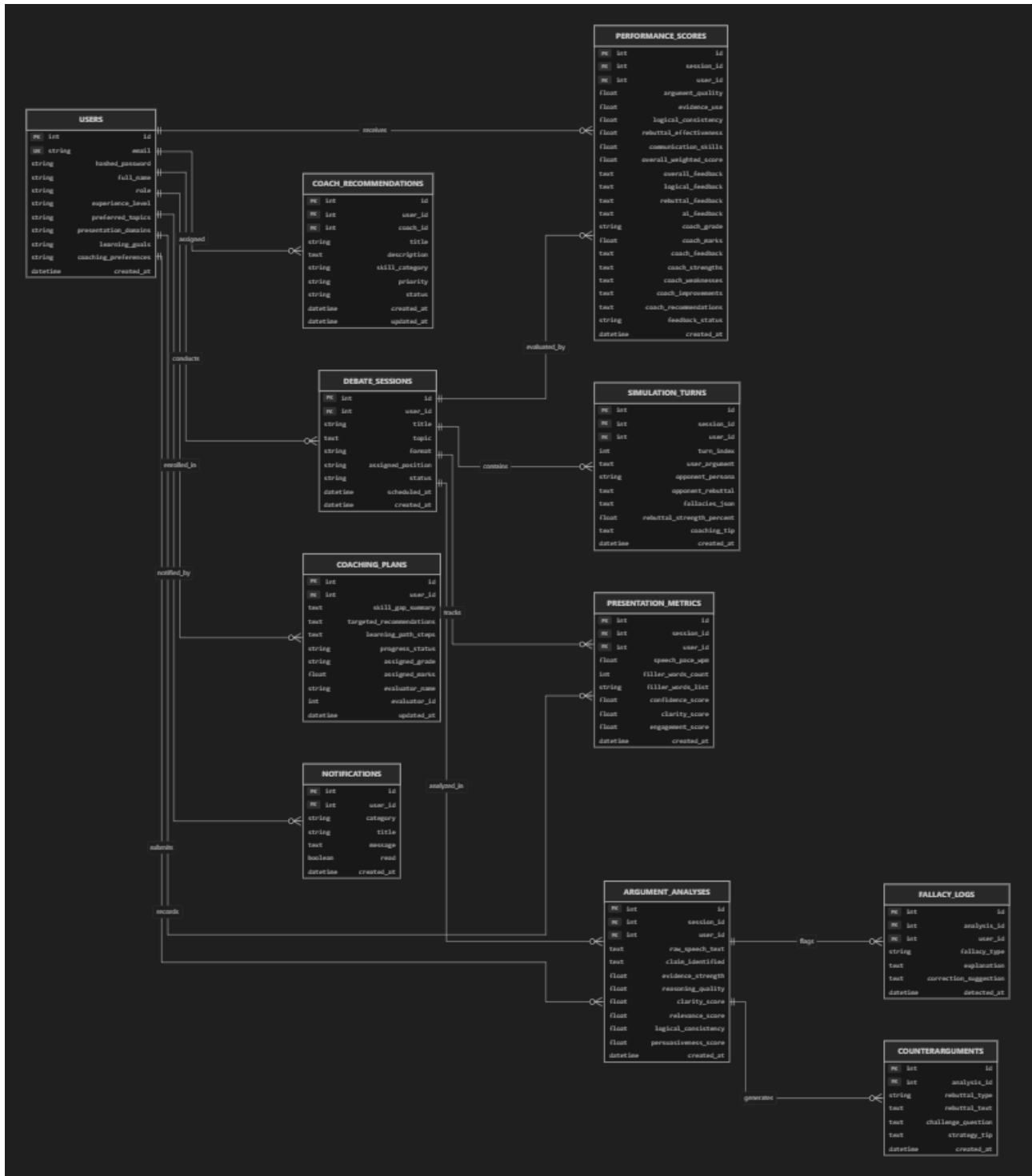


Figure 3 — Entity relationship diagram: 10 SQLAlchemy models and their foreign-key relationships.

6. End-to-end system sequence workflows

6.1 Debate simulation, real-time interception & PDF generation

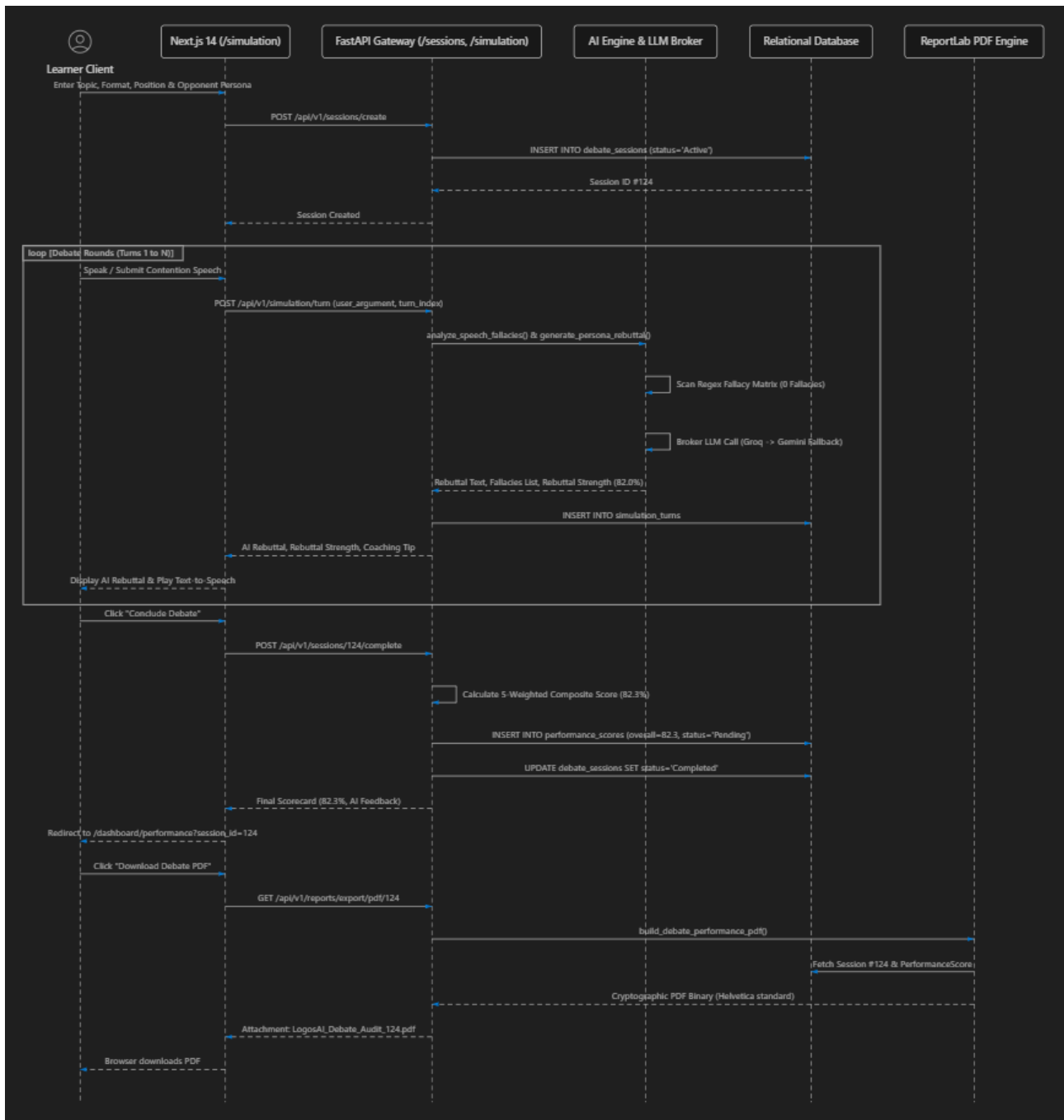


Figure 4 — Sequence: session creation, multi-turn simulation, score calculation, and cryptographic PDF export.

6.2 Coach student progress monitoring & feedback submission

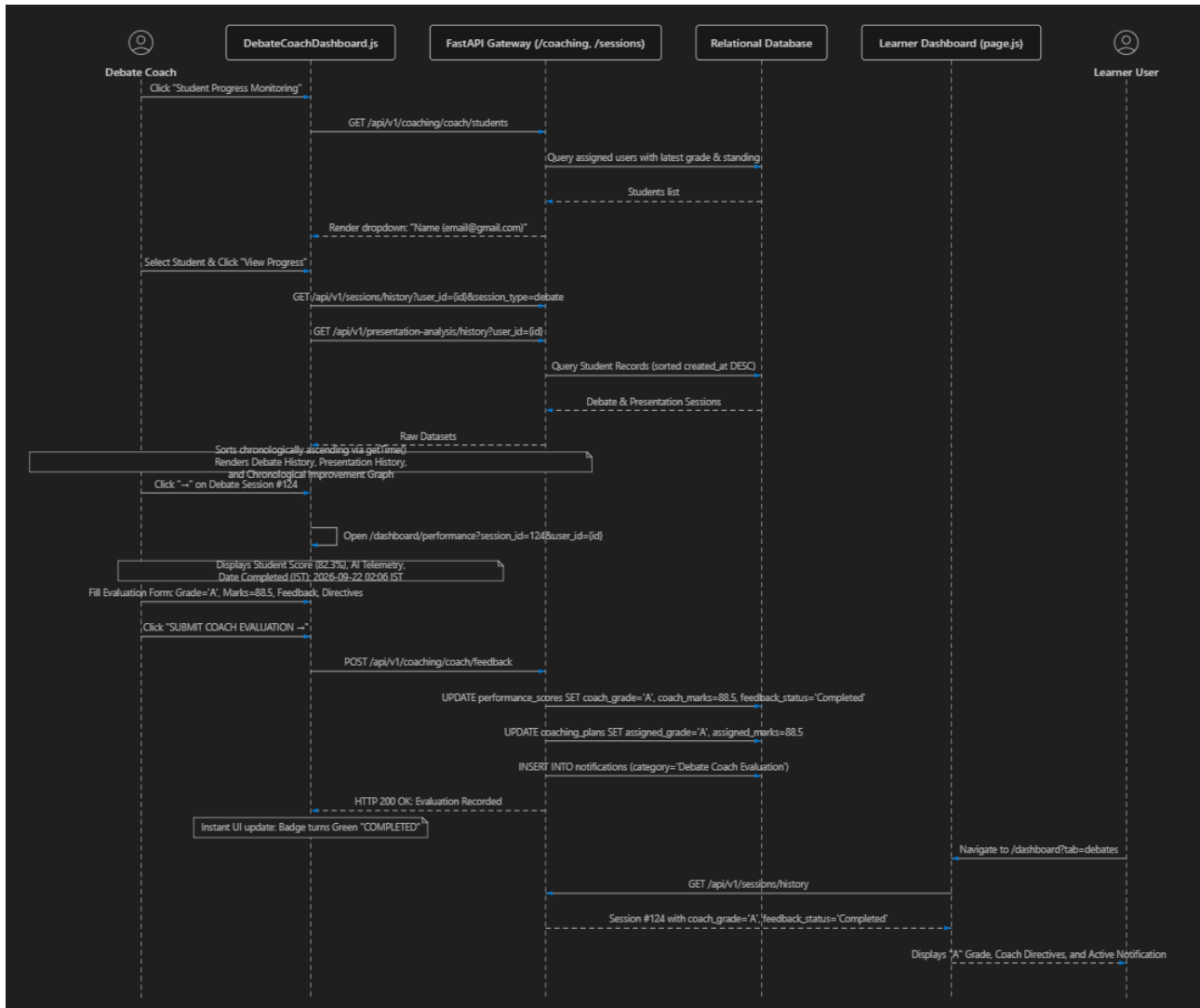


Figure 5 — Sequence: coach roster review, student progress inspection, evaluation submission, and learner-side notification.

7. Infrastructure, deployment & container topology

The complete platform runs in a containerized environment managed via Docker Compose.

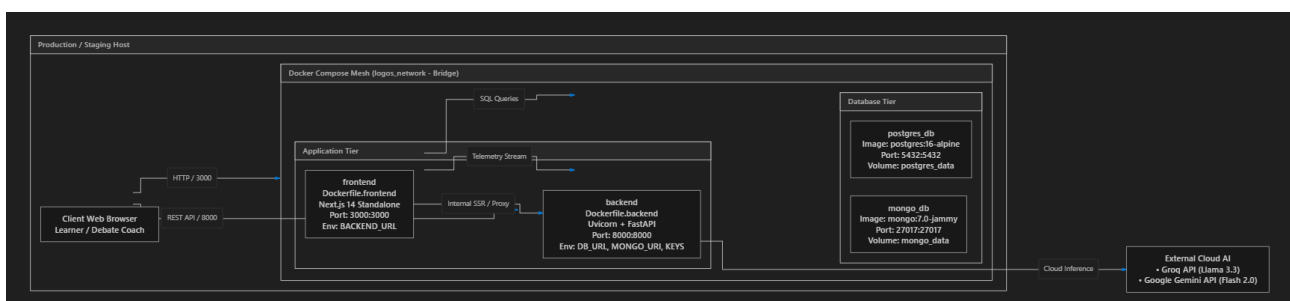


Figure 6 — Docker Compose mesh: frontend, backend, postgres_db and mongo_db on the logos_network bridge, with external Grog/Gemini cloud inference.

7.1 Container specifications

Service	Image base	Internal port	Host port	Storage volume	Healthcheck
postgres_db	postgres:16-alpine	5432	5432	postgres_data:/var/lib/postgresql/data	pg_isready -U postgres (10s, 5 retries)
mongo_db	mongo:7.0-jammy	27017	27017	mongo_data:/data/db	mongosh --eval "db.adminCommand('ping')"
backend	Python 3.11 Slim	8000	8000	Bind-mount ./backend, ./ai-ml	curl -f http://localhost:8000/health
frontend	Node 18 Alpine	3000	3000	Bind-mount ./frontend	curl -f http://localhost:3000/api/health

8. Architectural integrity checklist

[✓] Frontend routes verified: /, /login, /signup, /dashboard, /dashboard/performance, /simulation, /presentation-analysis, /argument-analysis, /fallacy-detector, /counter-argument.

[✓] Backend routers verified: 12 FastAPI microservices (auth, sessions, simulation, presentation_analysis, argument_analysis, fallacy_detection, counterarguments, coaching, scoring, dashboards, reports, notifications).

[✓] AI multi-tier orchestration: Groq Llama 3.3 70B → Gemini 2.0 Flash → local deterministic regex/FAISS engine.

[✓] Acoustic engine verified: WPM cadence, filler taxonomy (9 phrases), confidence and clarity formulas.

[✓] Data persistence verified: 10 SQLAlchemy models matching all foreign keys and cascades in models.py.

[✓] Chronological graphs verified: getTime(a) – getTime(b) chronological sort ensuring round 1 is oldest and round N is newest, with explicit IST labeling.

[✓] Coach workflow verified: roster overview, student progress dropdown "{name} ({email})", authentic learner dashboard view, and Pending → Completed feedback lifecycle.

[✓] PDF text extraction verified: ReportLab PDF rendering matching exact calculated scores and database values.